CONFIDENCE CALIBRATION REFERENCE

Jev vs. LLM: Confidence Calibration

Built for **confidence-audit**, a finance calibration harness
Jev (TypeSafe) vs. Claude Haiku 4.5, verbalized confidence

MOCK DATA: every number on this page comes from this repo's deterministic mock clients, which return a *constant* confidence by design. It validates the measurement pipeline end to end. It is not yet a finding about real Jev or real Claude. See page 3.

Jev (TypeSafe System One)

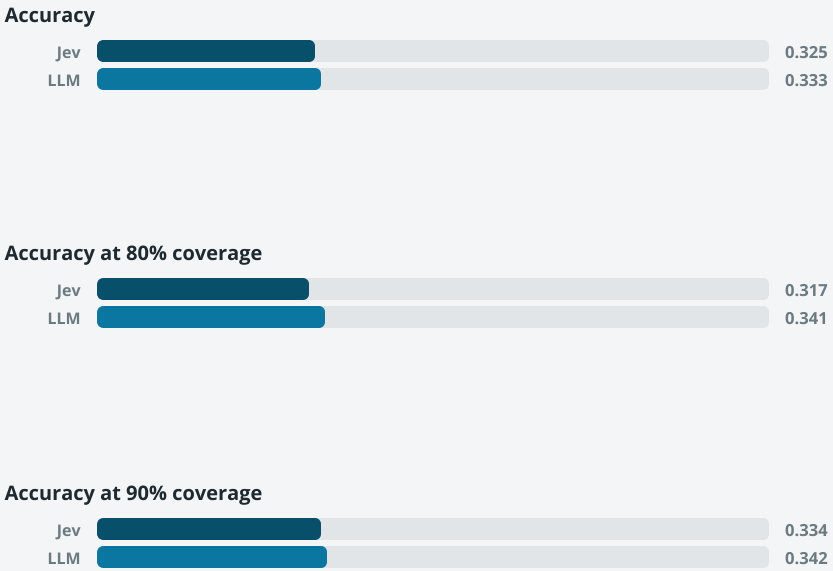
Native confidence from DecisionClient.choice(): a bounded pick from a fixed option list

LLM: Claude Haiku 4.5

Verbalized confidence: asked to self-report a 0–100 confidence alongside its label

Accuracy & coverage

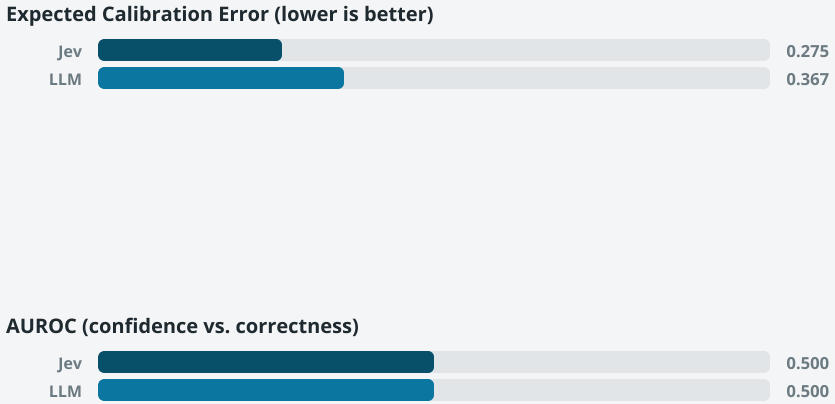
financial_sentiment, test split, n=679 (runs/20261004T200426Z/metrics.json)



All three near 1/3, expected on a 3-class task when predictions are an input-hash, uncorrelated with the true label.

Calibration & discrimination

Same split. ECE: lower is better. AUROC: 0.5 = no discrimination, 1.0 = perfect.



AUROC = 0.500 for both is the mathematical signature of a constant confidence score: there's nothing to rank by yet.

Source: confidence-audit (this repo), scripts/evaluate.py, runs/20261004T200426Z/metrics.json. Mock Jev + mock LLM clients, fixed seed, deterministic.

Pros & Cons by Approach

Jev

TypeSafe System One (src/harness/models/decision_jev.py)

PROS

- Returns a confidence natively with every decision, no prompting required
- Structurally bounded to the given option list; can never answer outside it
- Fast: a single real smoke-test call returned in ~0.2s
- Already used in Project 1 to gate retrieval and answer display

CONS

- Weak at numbers, counting, and literal detail by design (Project 1's own finding)
- Only bounded judgments, no open-ended writing
- Real calibration on this harness's tasks is still unmeasured end to end

BEST FIT

High-volume bounded decisions already embedded in app logic: the risk-topic tagging, relevance filtering, and groundedness gates Project 1 already runs on every paragraph.

LLM (Claude Haiku 4.5)

Verbalized confidence (src/harness/models/llm_client.py)

PROS

- Flexible: the same client answers any classification task via prompting
- Can be prompted to justify an answer, not just emit a bare label
- No fixed option-list infrastructure needed to stand up a new task

CONS

- Confidence is self-reported, not trained against a scoring rule: not the same thing as a calibrated probability (documented limitation, see REPORT.md)
- Slower and costlier per call than Jev at volume
- Needs a parser for its JSON output; a malformed response is a retryable failure

BEST FIT

Tasks where a fixed option list doesn't fit, or as an independent second opinion to sanity-check Jev's decisions on the same input.

Sources: this repo's src/harness/models/{decision_jev,llm_client}.py and REPORT.md's Limitations section.

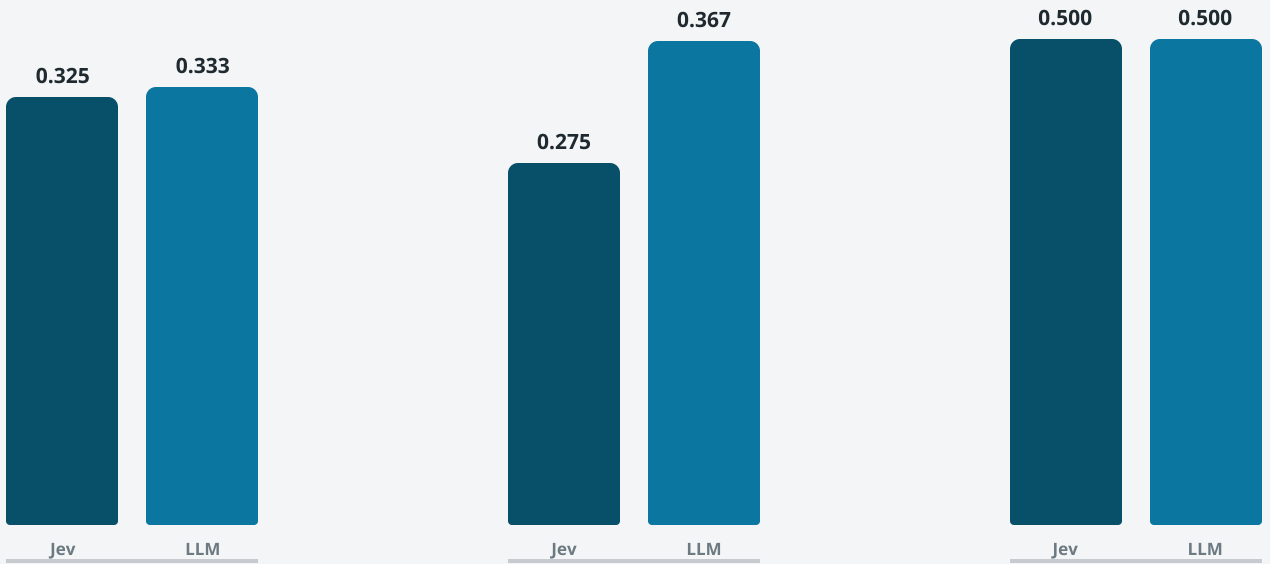
CONFIDENCE CALIBRATION REFERENCE

Status: Mock-Validated, Real Results Pending

The pipeline runs end to end and the numbers below are **real output from this repo**, but both clients return a **constant** confidence by design (0.6 for the mock Jev, 0.7 for the mock LLM), so nothing here is a real calibration finding yet. AUROC lands on exactly 0.500 for both: the mathematical signature of a score with no variation to rank by. The real **JevDecisionClient** is already wired up and smoke-tested: a single real call against each task returned a sensible, varying answer in well under a second, so what remains is a full real run, plus hand-labeling the 10-K gold sample before that task has any gold labels at all.

Where the two clients land today

Same financial_sentiment test-split numbers as page 1, reproduced as a vertical comparison



Bar heights scaled independently per metric for readability (ECE and AUROC are both 0–1 but not comparable to each other). Mock data, see banner on page 1.